Informed Search

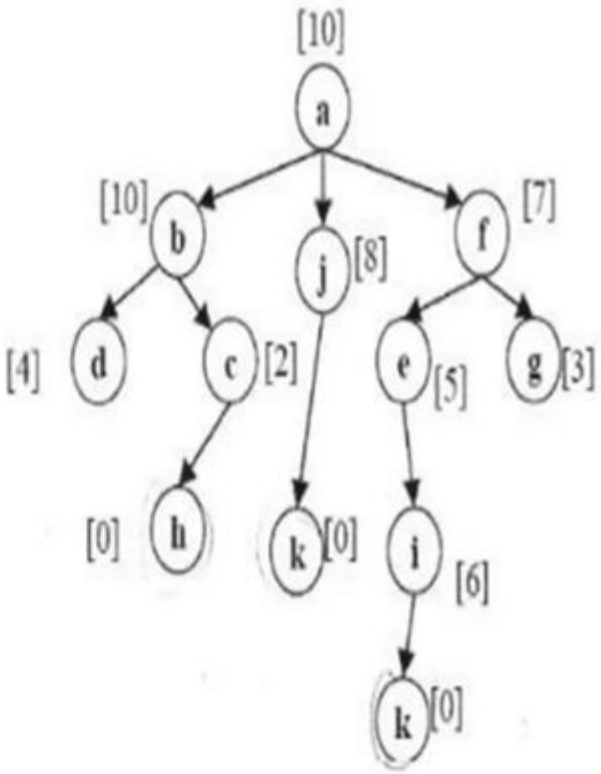
Difference Between Informed and Uninformed Search

BASIS FOR COMPARISON	INFORMED SEARCH	UNINFORMED SEARCH
Basic	Uses knowledge to find the steps to the solution.	No use of knowledge
Efficiency	Highly efficient as consumes less time and cost.	Efficiency is mediatory
Cost	Low	Comparatively high
Performance	Finds solution more quickly	Speed is slower than informed search
Algorithms	Heuristic search	Blind search

HILL CLIMBING SEARCH

1. Evaluate the initial state. If it is a goal state then return it and quit. Otherwise, continue with initial state as current state.
2. Loop until a solution is found or there are no new operators left to be applied in current state:
   - Select an operator that has not yet been applied to the current state and apply it to produce a new state
   - Evaluate the new state:
     * if it is a goal state, then return it and quit
     * if it is not a goal state but better than current state then make new state as current state
     * if it is not better than current state then continue in the loop

- Goal state : h and k
- Local minimum: a -> f -> g
- Solution:
  - a, j, k
  - a, f, e, i, k



Steepest-Ascent Hill Climbing (Gradient Search)

- Considers all the moves from the current state.
- Selects the best one as the next state.

Simple Hill Climbing moves to the *first* neighboring state that shows improvement, while **Steepest Hill Climbing** evaluates *all* neighbors and selects the one with the greatest improvement.

Steepest-Ascent Hill Climbing (Gradient Search)

1. Evaluate the initial state. If it is a goal state then return it and quit. Otherwise, continue with initial state as current state.

2. Loop until a solution is found or a complete iteration produces no change to current state:

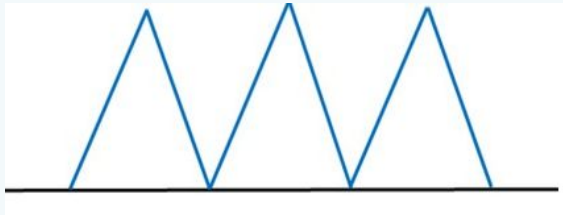
   - let SUCC be a state such that any possible successor of the current state will be better than SUCC (the worst state).
   - For each operator that applies to the current state do:
     * Apply any operator and generate new state
     * evaluate the new state:
       * if it is a goal state, then return it and quit
       * if it is not a goal state, compare it to SUCC.
         If it is better than set SUCC to this state
         If it is not better than leave SUCC alone
     * if SUCC is better than the current state then set the current state to SUCC.

DRAWBACK

- Local maxima: A local maximum is a peak that is higher than each of its neighboring states, but lower than the global maximum. Once a local maximum peak the algorithm is halt even though the solution may be far from satisfactory.

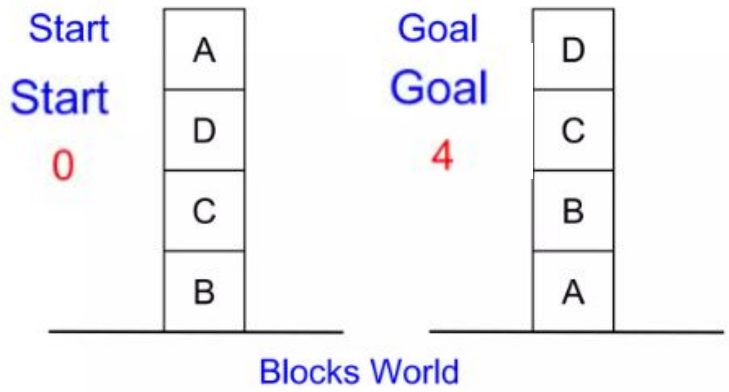
- Plateau: A plateau is an area of the state space landscape where the evaluation function is flat. It can be a flat local maximum , from which no uphill exit exists .The search will conduct a random walk.

Ridge A ridge is a special form of the local maximum. It has an area which is higher than its surrounding areas, but itself has a slope, and cannot be reached in a single move.



Ways Out

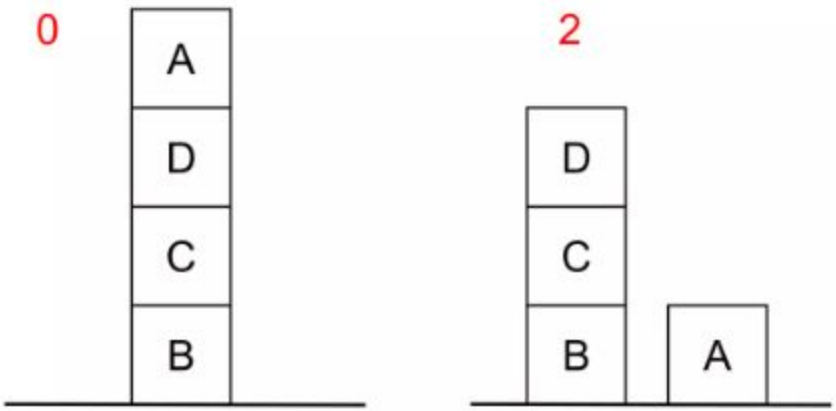
- **Local maximum :**Backtrack to some earlier node and try going in a different direction.
- **Plateau:** Here make a big jump to some direction and try to get to new section of the search space.
- **Ridge:** Here apply two or more rules before doing the test i.e., moving in several directions at once.
- Hill climbing is a local method:
  Decides what to do next by looking only at the "immediate" consequences of its choices.
- Global information might be encoded in heuristic functions.

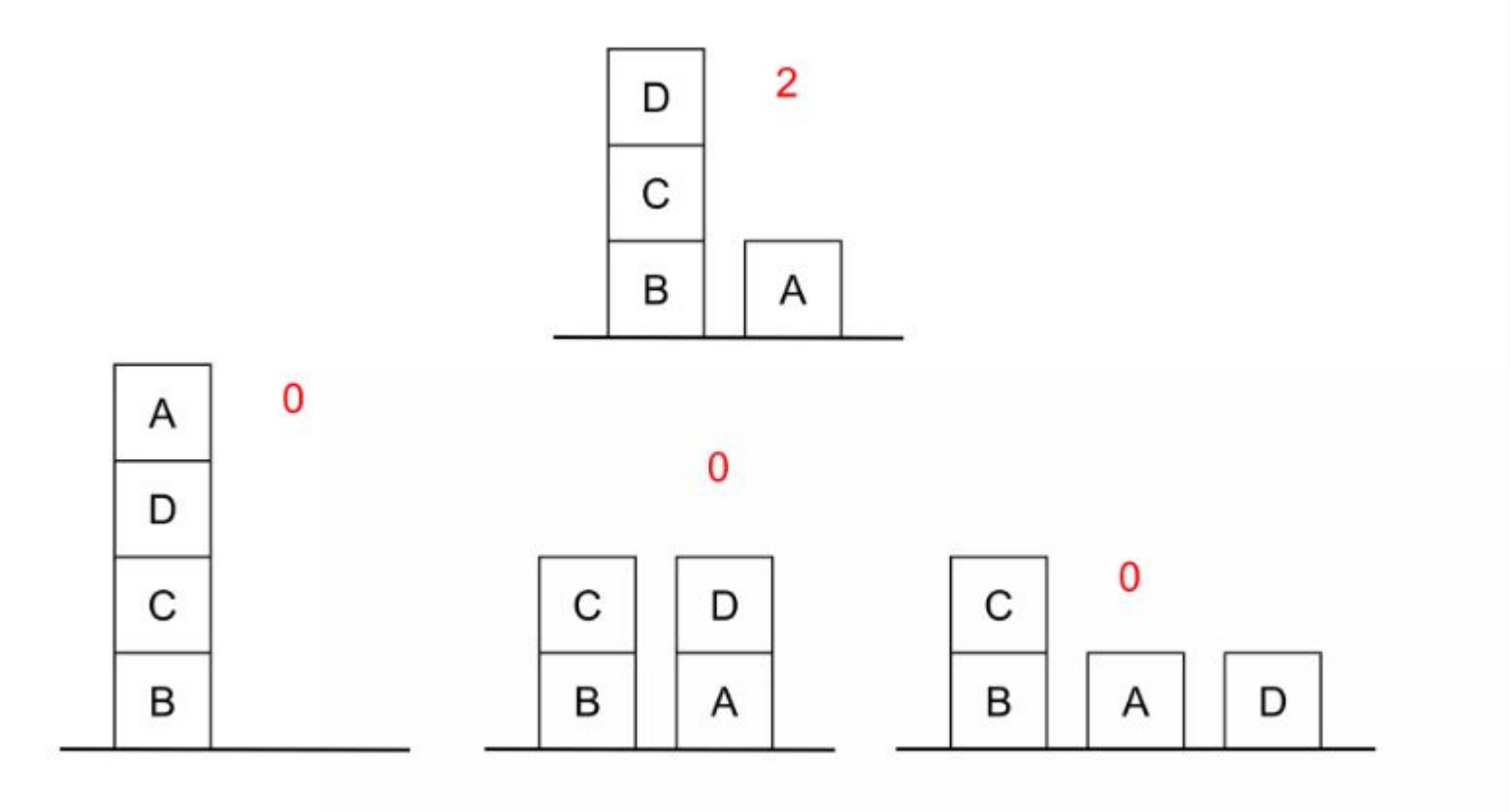


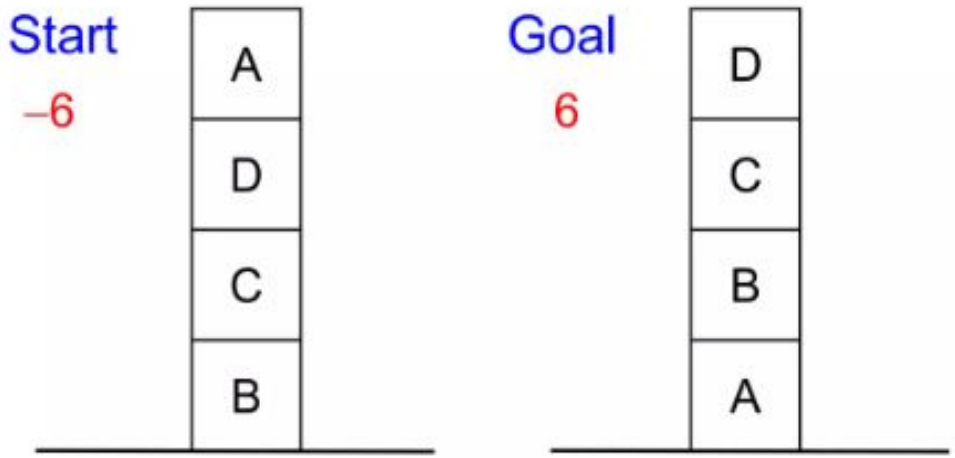
Local heuristic:

+1 for each block that is resting on the thing it is supposed to be resting on.

–1 for each block that is resting on a wrong thing.




D
C
B
A
2
A
D
C
B
0
C
D
B
A
0
C
B
A
D
0

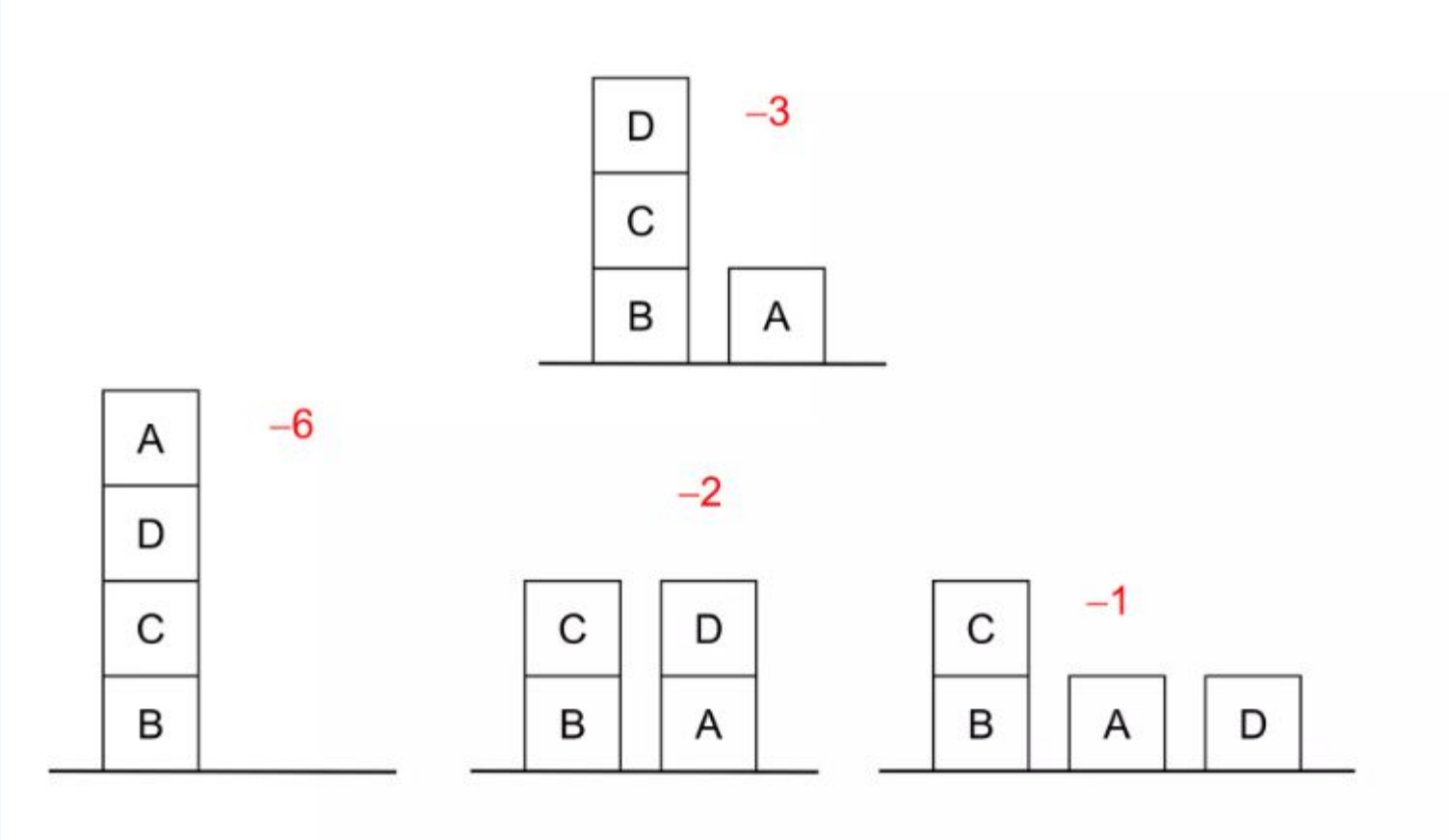


Blocks World

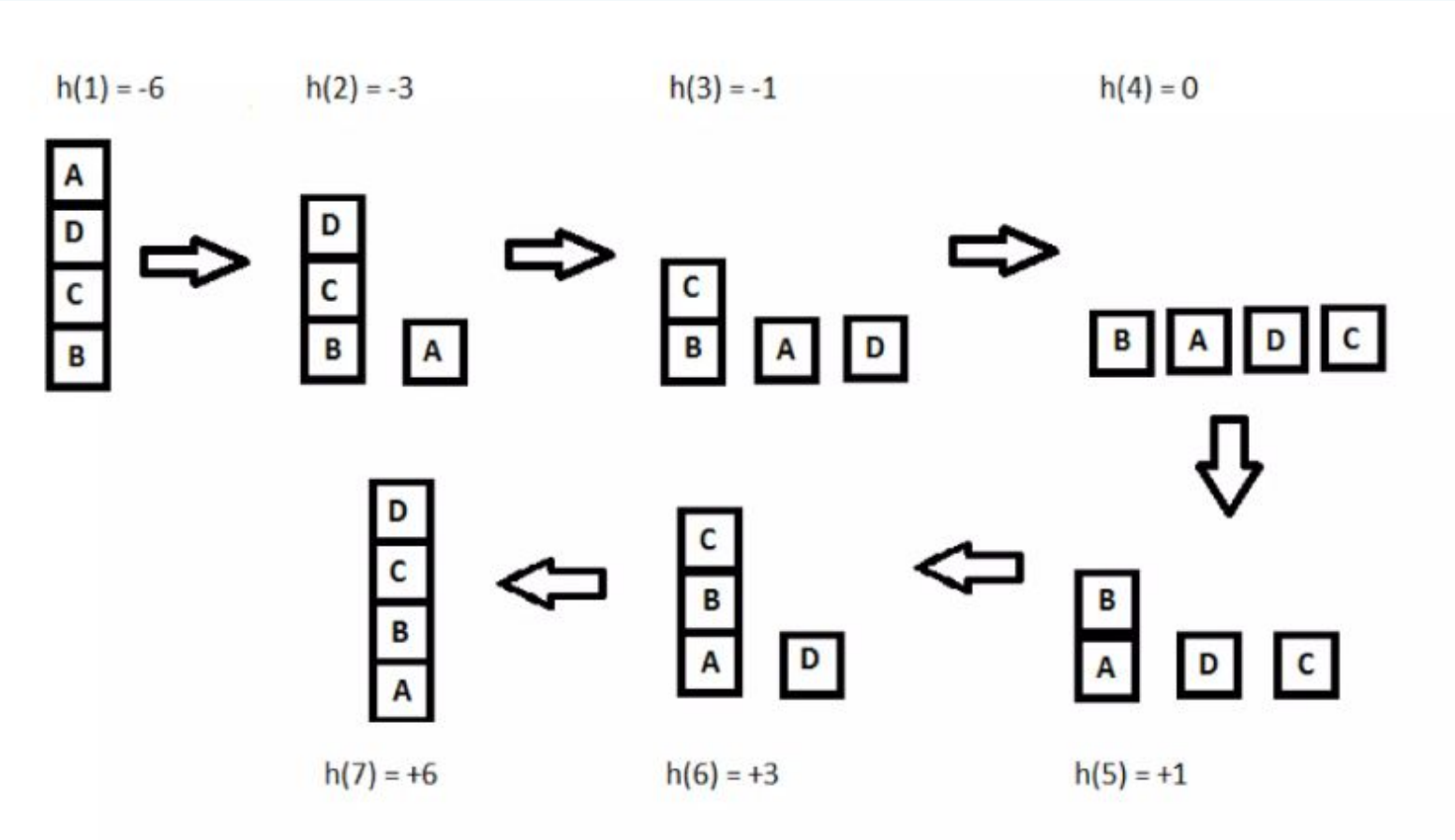
Global heuristic:

For each block that has the correct support structure: +1 to every block in the support structure.

For each block that has a wrong support structure: –1 to every block in the support structure.


D
C
B
A
–3
A
D
C
B
–6
–2
C
D
B
A
C
B
A
D
–1

Solution



Simulated Annealing

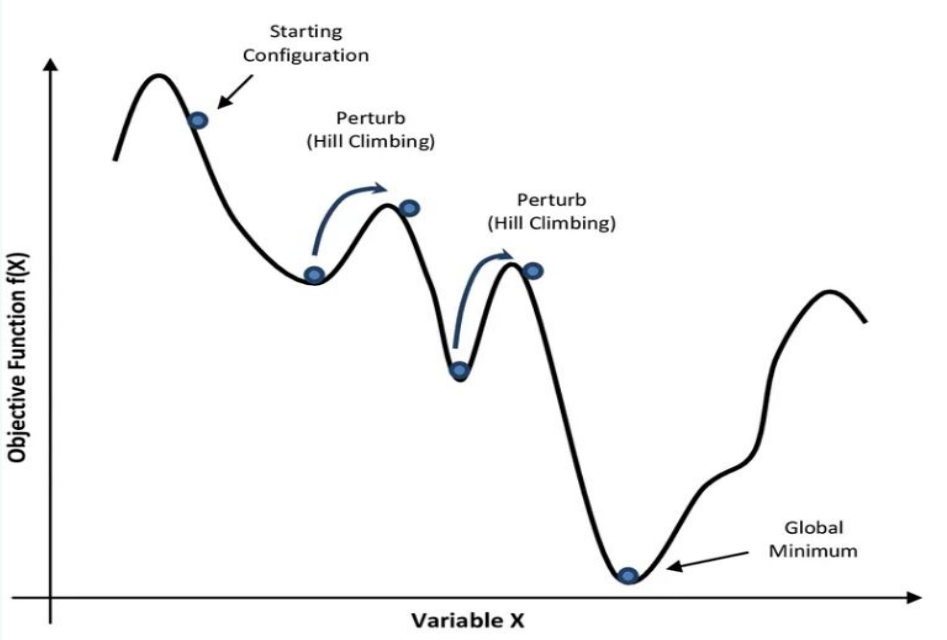
- Optimum solution in large solution space is difficult, as filled with local optima.
- ***Simulated Annealing*** (**SA**), inspired by the physical process of annealing in metallurgy.
- Degree of randomness in the search process (heat), which decreases over time (cooling schedule) to refine the solution.
- It works by starting with a high "temperature," allowing random moves (temp, heat) to worse solutions to explore broadly, then gradually "cooling" (reducing temp, heat) the system to decrease the probability of accepting bad moves, eventually settling into a stable, low-energy (optimal) state.
- The probability decreases exponentially with the badness of move, which is the amount ΔE by which the solution is worsened(i.e. energy is increased).
- SA excels in escaping these local minima by introducing **controlled randomness** in its search, allowing for a more thorough exploration of the solution space probabilistically and find a near-optimal solution (global optimum).

- A variation of hill climbing in which, at the beginning of the process, some downhill moves may be made.

- To do enough exploration of the whole space early on, so that the final solution is relatively insensitive to the starting state.

SA is robust for non-convex problems, while Gradient Descent (GD) (GD) greedily follows the steepest downhill path, making it fast for convex problems but prone to getting stuck in local minima.

- **Exploration vs. Exploitation**: SA explores widely (high temp) then exploits (low temp); GD primarily exploits the current gradient.



The algorithm is basically a hill-climbing except instead of picking the best move, it picks a random move.

If the selected move improves the solution, then it is always accepted. Otherwise, the algorithm makes the move anyway with some probability less than 1.

How Simulated Annealing Works

- **Initialization:** Begin with an initial solution *So* and an initial temperature *To*.
- **Neighborhood Search**: At each step, a new solution *S'* is generated by making a small change (or perturbation) to the current solution *So*.
- **Objective Function:** The new solution S' is evaluated using the objective function. If S' provides a better solution than S, it is accepted as the new solution.
- **Acceptance Probability**: If S' is worse than S, it may still be accepted with a probability based on the temperature and the difference in objective function values. The acceptance probability is given by:

$$p(\text{accept}) = e^{-\Delta E/\alpha T};$$

generate a random number *r*; if r < p accept the new solution

- Cooling Schedule: After each iteration, the temperature is decreased according to a predefined cooling schedule, which determines how quickly the algorithm converges.
- Termination: The algorithm continues until the system reaches a low temperature (i.e., no more significant improvements are found), or a predetermined number of iterations is reached.

Simulated Annealing

1. Evaluate the initial state. If it is a goal state then return it and quit. Otherwise, continue with initial state as current state.
2. Initialize best-so-far to the current state.
3. Initialize T according to the annealing schedule.
4. Loop until a solution is found or there are no new operators left to be applied in current state:
   1. Select an operator that has not yet been applied to the current state and apply it to produce a new state
   2. Evaluate the new state, compute:

      ΔE = Val(current state) - Val(new state)

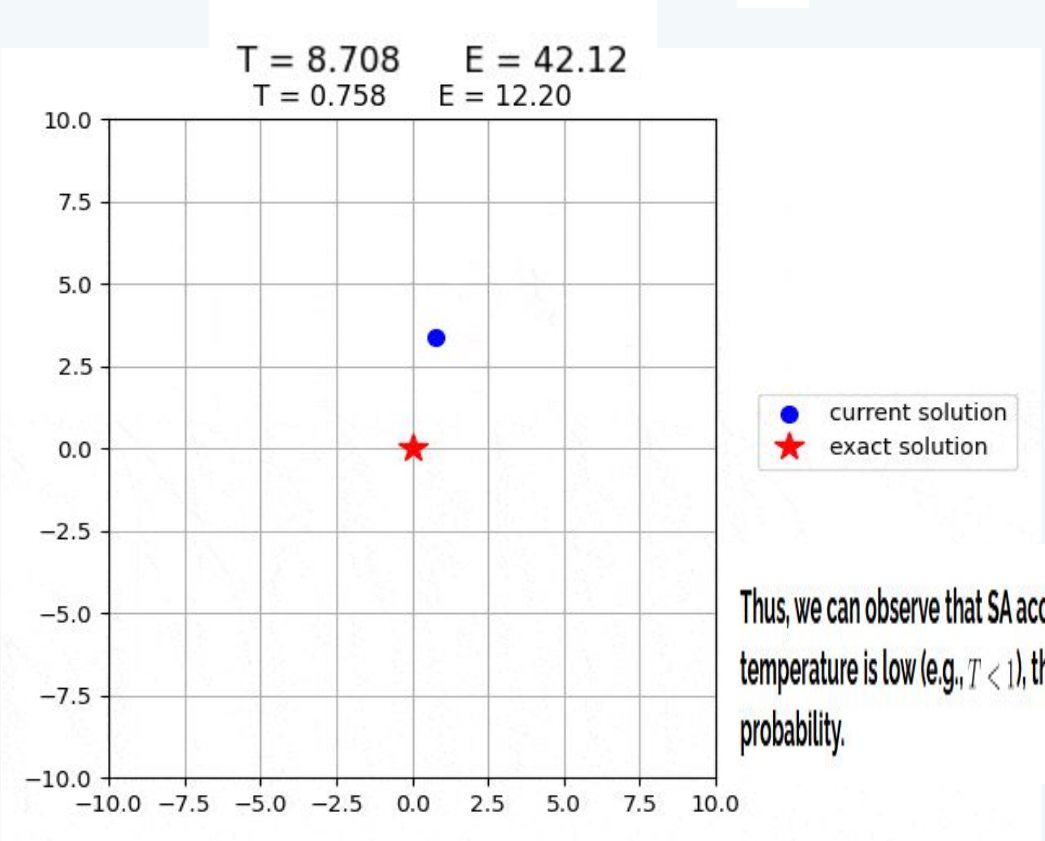
      * if new state is a goal state, then return it and quit

      * if it is not a goal state but better than current state then make new state as current state

      * if it is not better than current state then make new state as current state with probability p` = e ^ -$\Delta$E/T. This step is usually implemented by invoking a random number generator to produce a number in the range of [0,1]. If the number is less then p`, then the move is accepted ,otherwise do nothing.

   3. Revise T as necessary according to the annealing schedule.

- Minimization of the function $f(x, y) = x^2 + y^2$.
- Search space is a grid of size 101 × 101 placed in the square area defined by $x, y \in [-10, +10]$.
- Furthermore, we set the cooling rate $\alpha = 0.84$ and the initial solution (x, y) = (1, 3.5).
- Additionally, at each step, we generate a new solution by randomly shifting the current solution by ± 0.2 in x and y direction.



Thus, we can observe that SA accepts the worse solutions when the temperature is high. Conversely, when the temperature is low (e.g., $T < 1$), the algorithm is more selective, and better solutions are accepted with higher probability.

Heuristic Function

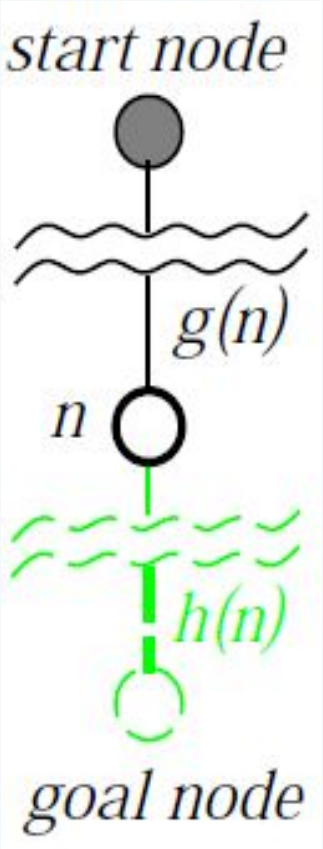
HEURISTIC INFORMATION:

Information about the problem (the nature of the states, the cost of transforming from one state to another , the promise of taking a certain path, and the characteristics of the goals).can sometimes be used to help guide the search more efficiently.

- Choice of heuristics depends on knowledge of the problem space
- An informed guess of the next step to be taken in solving a problem would prune the search space
- A heuristic may find a sub-optimal solution or fail to find a solution since it uses limited information
- In search algorithms, heuristic refers to a function that provides an estimate of solution cost

Evaluation Function

- Evaluation function f(n) = g(n) + h(n)
  - g(n) = exact cost of the cheapest path so far to reach *n*
  - h(n) = estimated cost of the cheapest path to goal from *n*
  - f(n) = estimated total cost of cheapest path through *n* (current state) to goal (measures goodness)
- h(n) takes a node as input, but, unlike g(*n*), it depends only on the state at that node.
- At goal state h(n) = 0



Best-first search (BFS)

- Depth-first search: not all competing branches having to be expanded.

- Breadth-first search: not getting trapped on dead-end paths.

$\Rightarrow$ Combining the two is to follow a single path at a time, but switch paths whenever some competing path look more promising than the current one.

The term **“Best First Search"** is a heuristic-driven search algorithm that prioritize exploring the most "promising" nodes, using an evaluation function to decide which among the various available nodes is the most promising (or ‘BEST’) before traversing to that node.

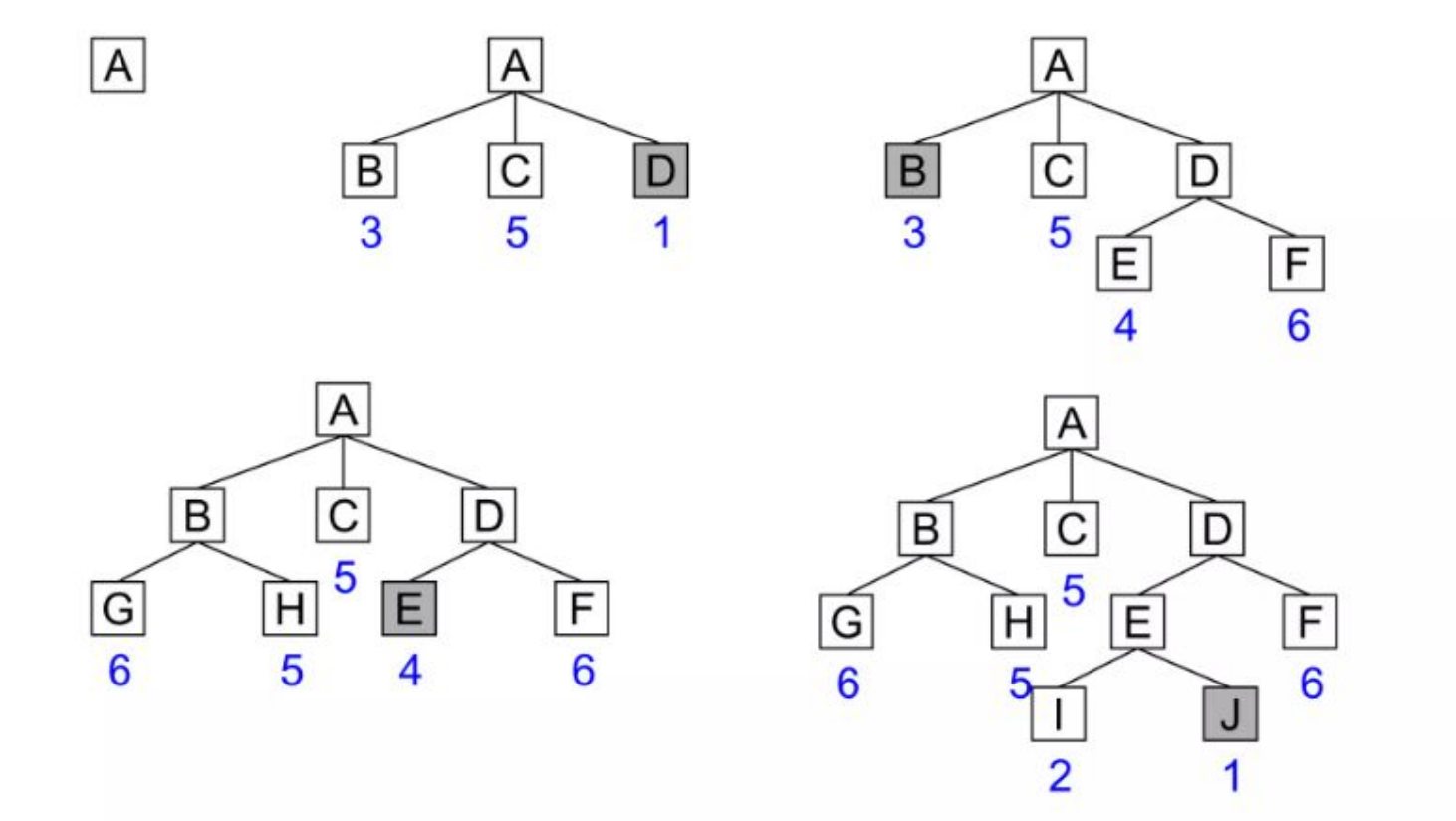
Variants of Best First Search

- Greedy Best First Search and A* Best First Search.
- **Greedy Best First Search:** Uses only a heuristic function h(n) to estimate the direct distance from the current node to the goal. Prioritizes nodes that seem closest to the goal.
- **A* Best First Search:** considers two factors: the cost of the path g(n) and the estimated cost to reach the goal h(n).
- The evaluation function $f(n)=g(n)+h(n)$ balances direct distance and travel cost for optimal path finding.

A* Search:

- f(n)=g(n)+h(n), search uses an "admissible" heuristic function *f* that takes into account the current cost *g*
- A heuristic function $f(n) = g(n) + h(n)$ is admissible if $h(n)$ **never** overestimates the cost to reach the goal.
- $f(n)$ never over-estimate the true cost to reach the goal state through node *n*.
- Always returns the optimal solution path

- In **Best-first search** a node is selected for expansion based on an **evaluation function, *f*(n).**
- The node with the *lowest evaluation is expanded first.*
- The choice of *f* determines the search strategy.



Greedy Best First Search

- Greedy Best-First Search is an informed search algorithm that selects the path to expand based on a heuristic evaluation of the current state's desirability in terms of reaching the goal.
- Unlike A* search, Greedy Best-First Search focuses solely on the estimated cost from the current state to the goal state without considering the cost incurred so far.
- ~ No Consideration of Path Cost: Greedy Best-First Search does not take into account the cumulative cost of the path traveled so far. It may choose a path that looks promising based on the heuristic alone, even if it has a higher actual cost.
- ~ Completeness and Optimality: Greedy Best-First Search is not guaranteed to be complete or optimal. It might get stuck in a local minimum and may not find the optimal solution.
- ~ Memory Usage: Like other uninformed and informed search algorithms, Greedy Best-First Search may require significant memory resources, especially in scenarios with large state spaces .

Strategy

- Once the goal is found the parent link will make it possible to recover the path to the goal.
- The list of successors will make it possible, if a better path is found to an already existing node, to propagate the improvement down to its successors.
- We will call a graph of this sort an **OR *graph,*** since each of its branches represents an alternative problem-solving path.
- To implement such a graph-search procedure, we will need to use two lists of nodes:

- OPEN: nodes that have been generated, but have not examined. This is organized as a priority queue in which the elements with the highest priority are those with the most promising value of the heuristic function
- CLOSED: nodes that have already been examined. It used, whenever a new node is generated, check whether it has been generated before.

Algorithm

1. Start with *OPEN* containing just the initial state.
2. Until a goal is found or there are no nodes left on *OPEN do:*
   I. Pick the best node on *OPEN*.
   II. Generate its successors.
   III. For each successor do:
      a. If it has not been generated before, evaluate it, add it to *OPEN, and record its* parent.
      b. If it has been generated before, change the parent if this new path is better than the previous one. In that case, update the cost of getting to this node and to any successors that this node may already have.

Informed Search

- Idea: use an evaluation function for each node
  - Estimate of desirability
- Expand most desirable unexpanded node

- Special cases:
  - Uniform Cost Search (uninformed; $f(n) = g(n)$, $h(n) = 0$)
  - Greedy (best-first) Search (informed; $f(n) = h(n)$, $g(n) = 0$))

8 Puzzle Problem

2	8	3
1	6	4
7		5

S

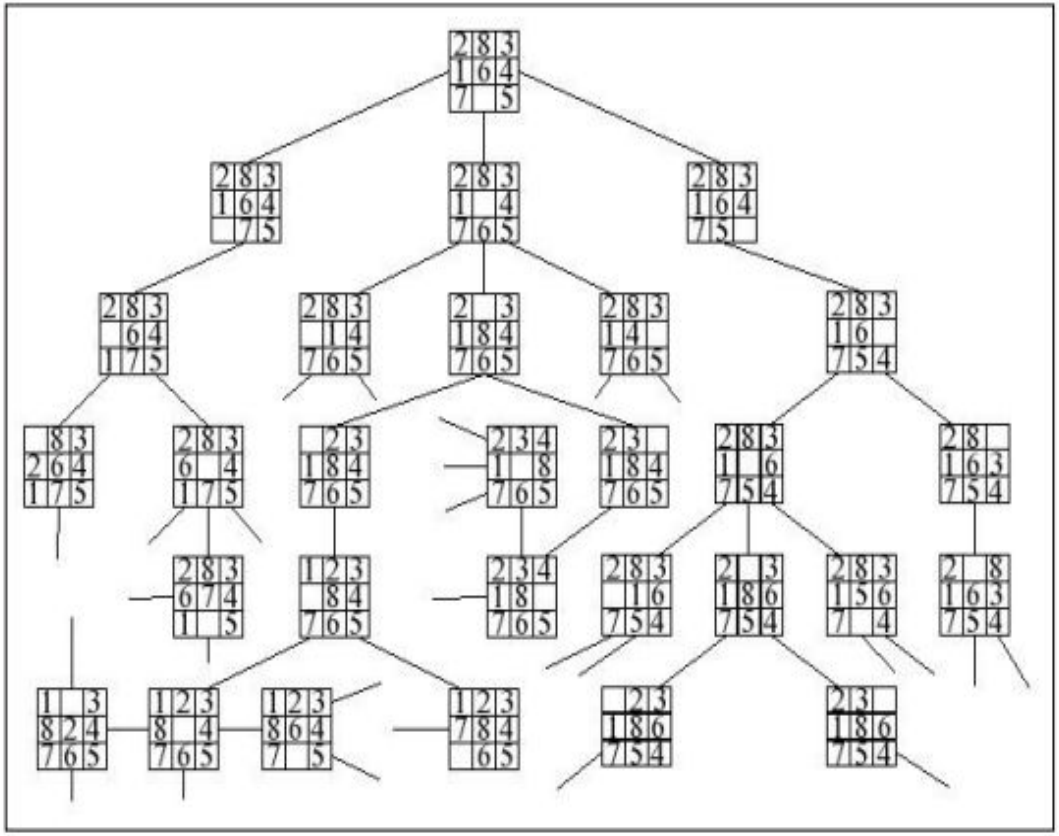
→

G

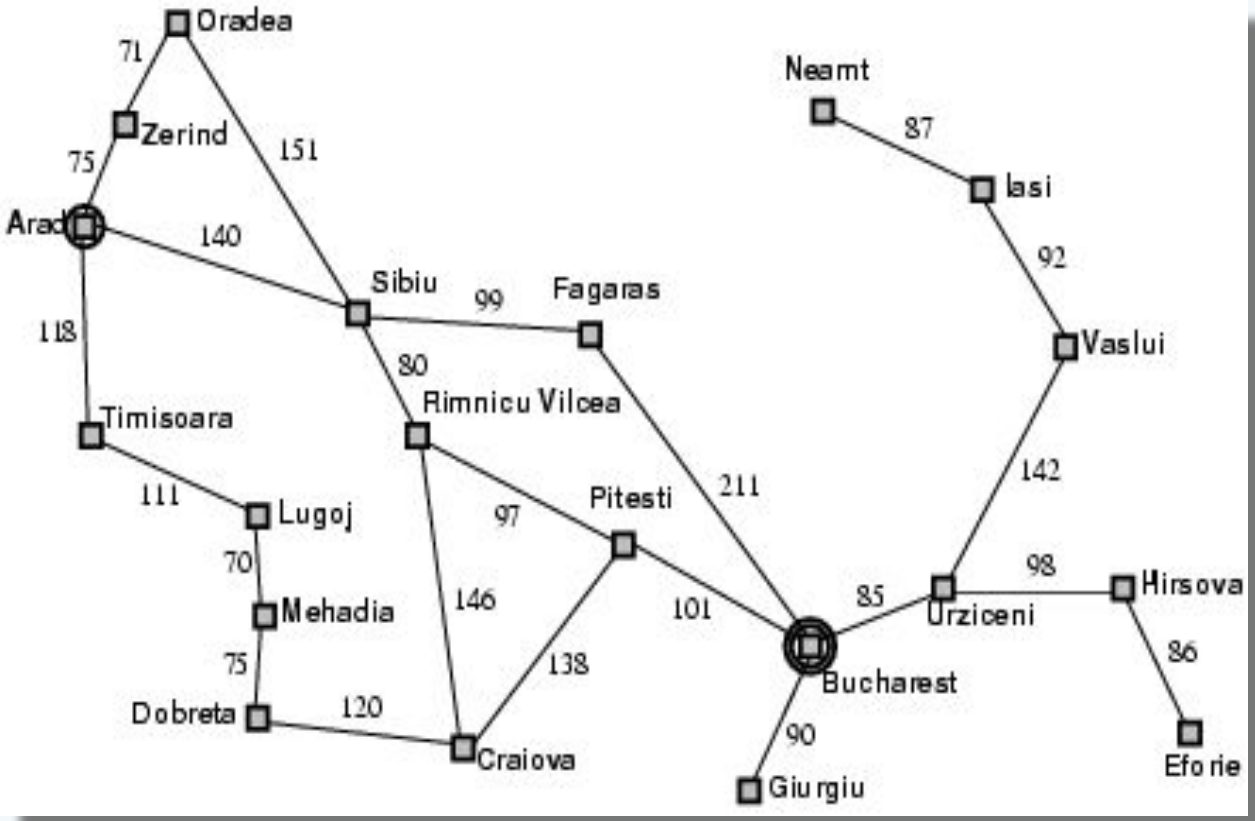
1	2	3
8		4
7	6	5

HEURISTIC 1: NUMBER OF MISPLACED TILES

HEURISTIC 2: SUM OF TILE DISTANCES FROM GOAL

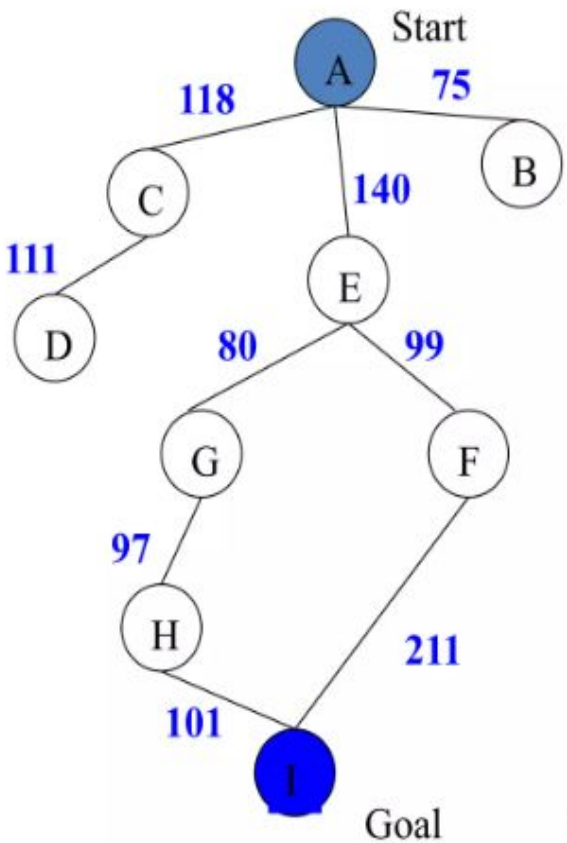


Romania - Step Costs in KM



Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Greedy Best First Search



State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0

***f(n) = h (n)* = straight-line distance heuristic**

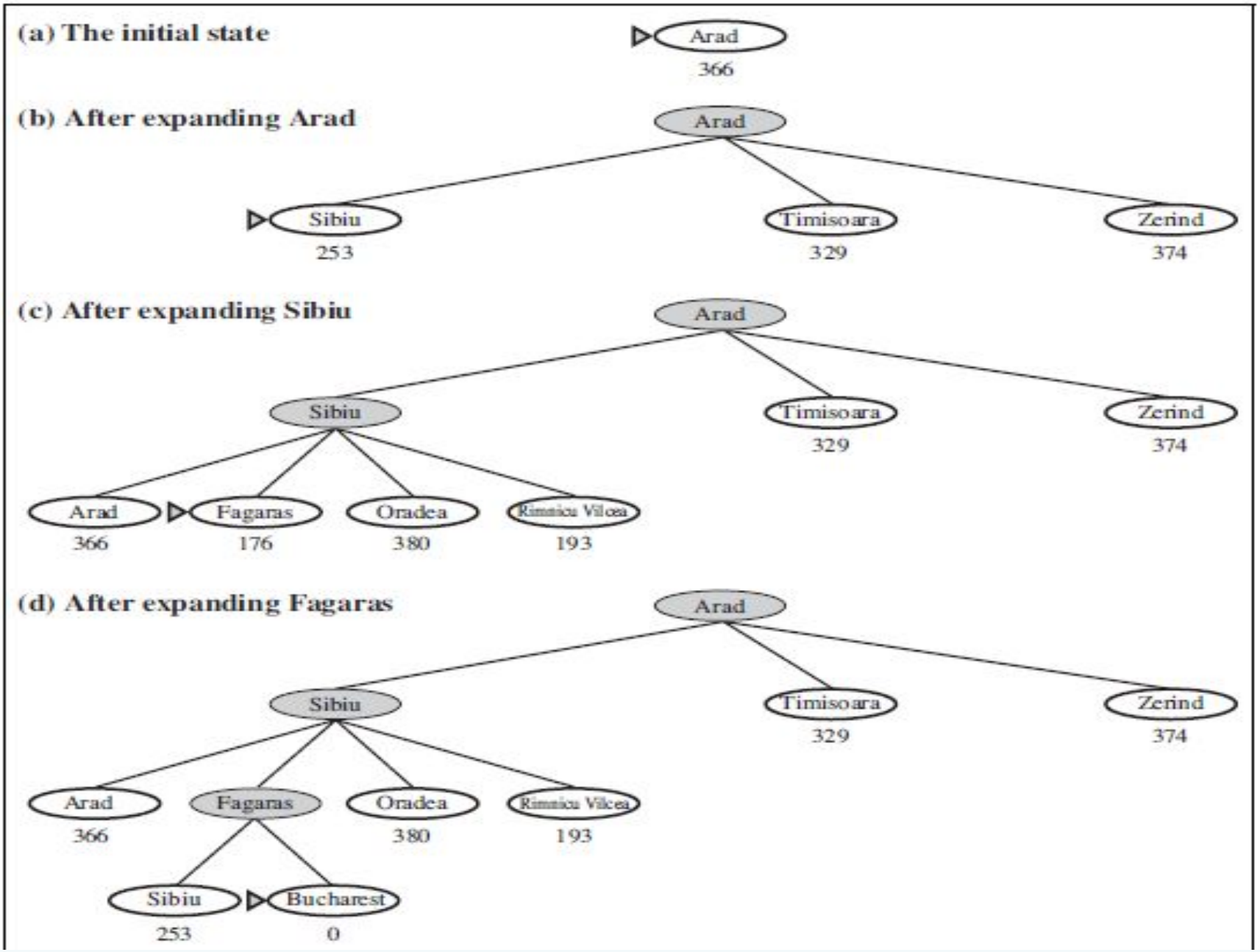
Greedy Best-First Search

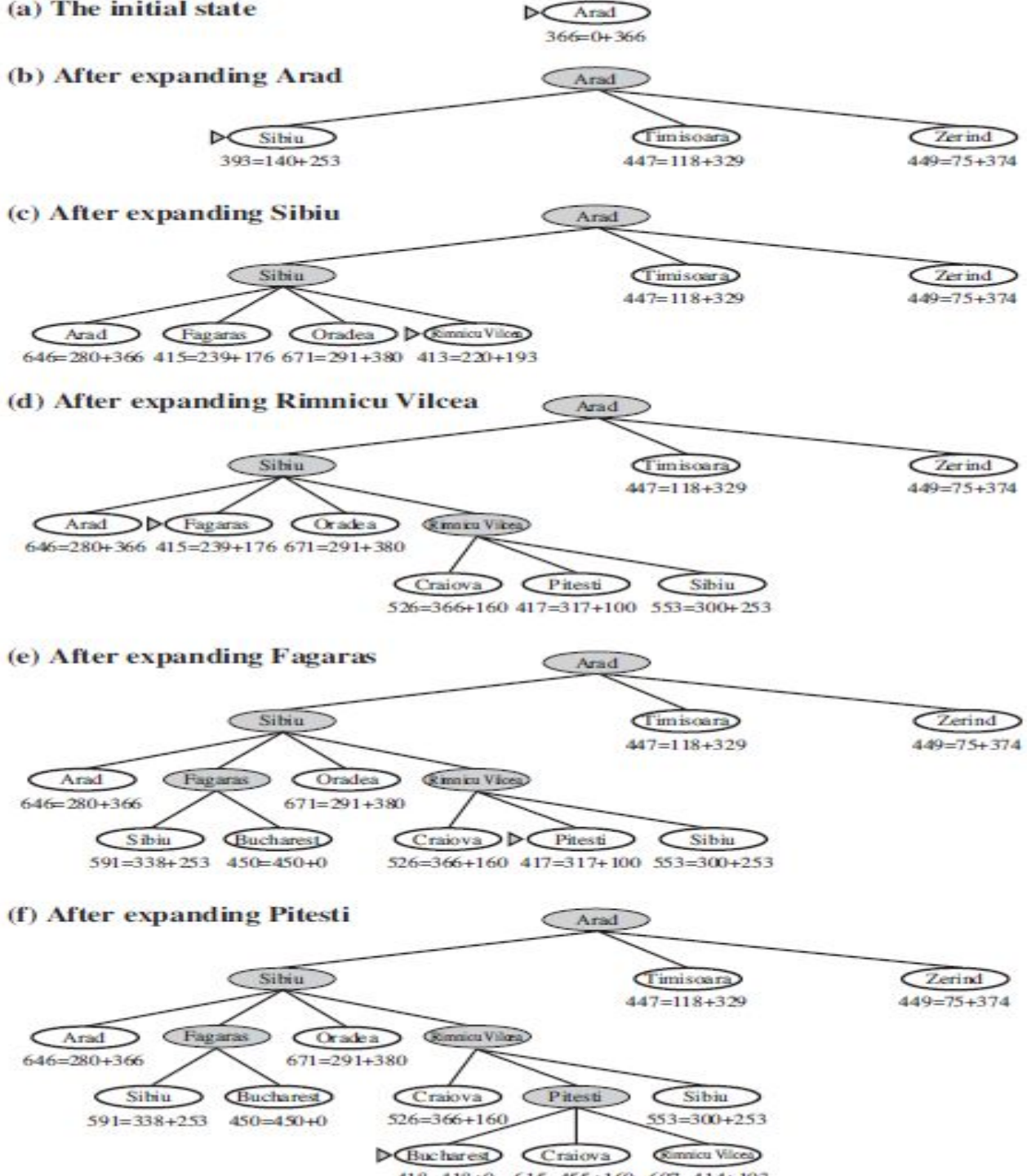
- Greedy best-first search tries to expand the node that is closest to the goal, on the grounds that this is likely to lead to a solution quickly.
- It evaluates nodes by using just the heuristic function (h(n); that is, f(n) = h(n).
  - Estimated cost of the cheapest path from n to a goal node
- E.g., $h_{SLD}(n)$ = straight-line distance from n to Bucharest

Route-finding problem

- If the goal is Bucharest, we need to know the straight-line distances to Bucharest.
- It takes a certain amount of experience to know that h_{SLD} is correlated with actual road distances and is, therefore, a useful heuristic.

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374


(a) The initial state
Arad
366
(b) After expanding Arad
Arad
Sibiu
253
Timisoara
329
Zerind
374
(c) After expanding Sibiu
Arad
Sibiu
Timisoara
329
Zerind
374
Arad
366
Fagaras
176
Oradea
380
Rimnicu Vilcea
193
(d) After expanding Fagaras
Arad
Sibiu
Timisoara
329
Zerind
374
Arad
366
Fagaras
Oradea
380
Rimnicu Vilcea
193
Sibiu
253
Bucharest
0

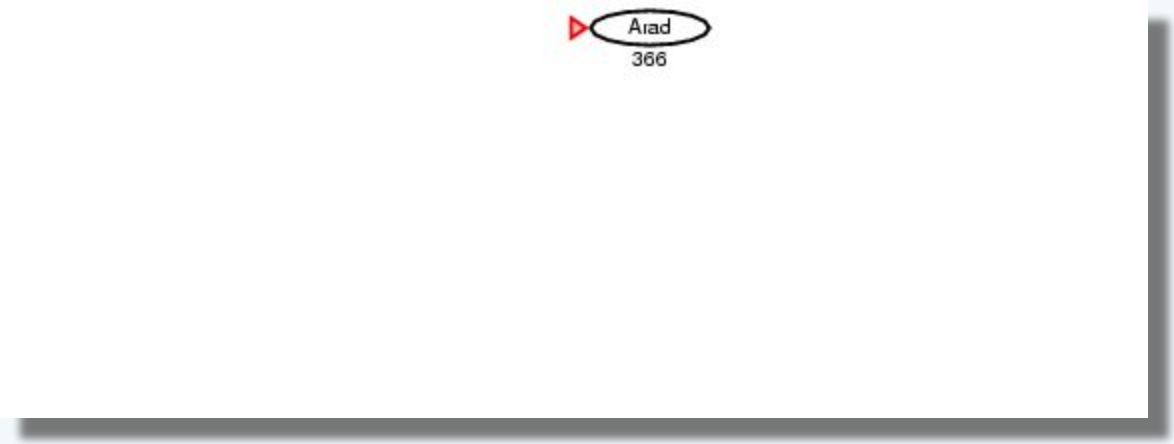

(a) The initial state
Arad
366=0+366
(b) After expanding Arad
Arad
Sibiu
393=140+253
Timisoara
447=118+329
Zerind
449=75+374
(c) After expanding Sibiu
Arad
Sibiu
Timisoara
447=118+329
Zerind
449=75+374
Arad
646=280+366
Fagaras
415=239+176
Oradea
671=291+380
Rimnicu Vilcea
413=220+193
(d) After expanding Rimnicu Vilcea
Arad
Sibiu
Timisoara
447=118+329
Zerind
449=75+374
Arad
646=280+366
Fagaras
415=239+176
Oradea
671=291+380
Rimnicu Vilcea
Craiova
526=366+160
Pitesti
417=317+100
Sibiu
553=300+253
(e) After expanding Fagaras
Arad
Sibiu
Timisoara
447=118+329
Zerind
449=75+374
Arad
646=280+366
Fagaras
Oradea
671=291+380
Rimnicu Vilcea
Sibiu
591=338+253
Bucharest
450=450+0
Craiova
526=366+160
Pitesti
417=317+100
Sibiu
553=300+253
(f) After expanding Pitesti
Arad
Sibiu
Timisoara
447=118+329
Zerind
449=75+374
Arad
646=280+366
Fagaras
Oradea
671=291+380
Rimnicu Vilcea
Sibiu
591=338+253
Bucharest
450=450+0
Craiova
526=366+160
Pitesti
553=300+253
Bucharest
Craiova
Rimnicu Vilcea

- It is not optimal, however: the path via Sibiu and Fagaras to Bucharest is 32 kilometers longer than the path through Rimnicu Vilcea and Pitesti.
- This shows why the algorithm is called "greedy"—at each step it tries to get as close to the goal as it can.

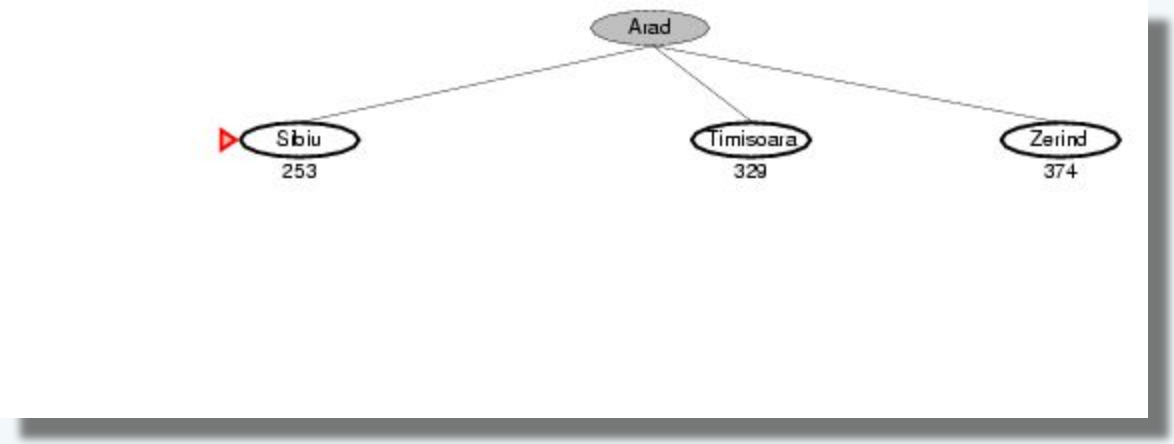
Consider the problem of getting from *Iasi* to *Fagaras*.

- The heuristic suggests that *Neamt* be expanded first because it is closest to *Fagaras*, but it is a dead end.
- The solution is to go first to Vaslui—a step that is actually farther from the goal according to the heuristic—and then to continue to Urziceni, Bucharest, and Fagaras.
- With a good heuristic function, however, the complexity can be reduced substantially.
- The amount of the reduction depends on the particular problem and on the quality of the heuristic.

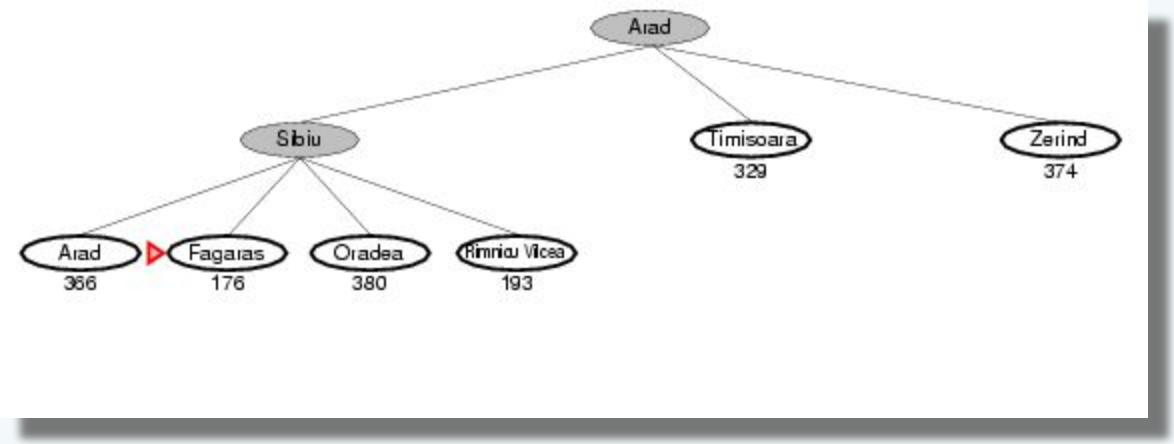
Greedy Best-First search example



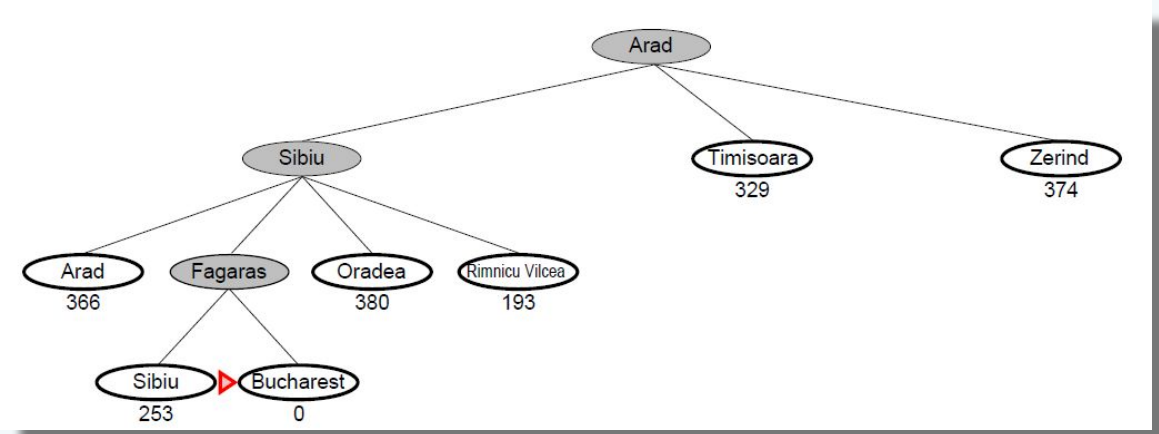
Greedy Best-First search example



Greedy Best-First search example



Greedy Best-First search example



Properties of Greedy Best-First search

- <u>Complete?</u> No – can get stuck in loops, e.g., with Oradea as goal and start from Iasi:
  - Iasi → Neamt → Iasi → Neamt →
    Complete in finite space with repeated state checking

- <u>Time?</u> $O(b^m)$, but a good heuristic can give dramatic improvement

- <u>Space?</u> $O(b^m)$ -- keeps all nodes in memory

- <u>Optimal?</u> No.

A* Algorithm

- A* search algorithm is designed to explore and navigate through graphs and used for path finding problem.
- A* algorithm for finding optimal paths.
- A* algorithm combines the best aspects of two other algorithms:

Dijkstra's Algorithm: This algorithm finds the shortest path to all nodes from a single source node.

Greedy Best-First Search: This algorithm explores the node that appears to be closest to the goal, based on a heuristic function.

A* Algorithm

- **Problem:** Trying to find the shortest route between two cities on a map.
- Dijkstra's algorithm would explore in all directions and Best-First Search might head straight toward the destination (potentially missing shortcuts).
- A* considers both:
- The distance already traveled from the start
- A smart estimate of the remaining distance to the goal
- This combination helps A* make informed decisions about which path to explore next, making it both efficient and accurate.

- The A* algorithm maintains two essential lists

Open list:

- Contains nodes that need to be evaluated
- Sorted by f(n) value (lowest first)
- New nodes are added as they're discovered

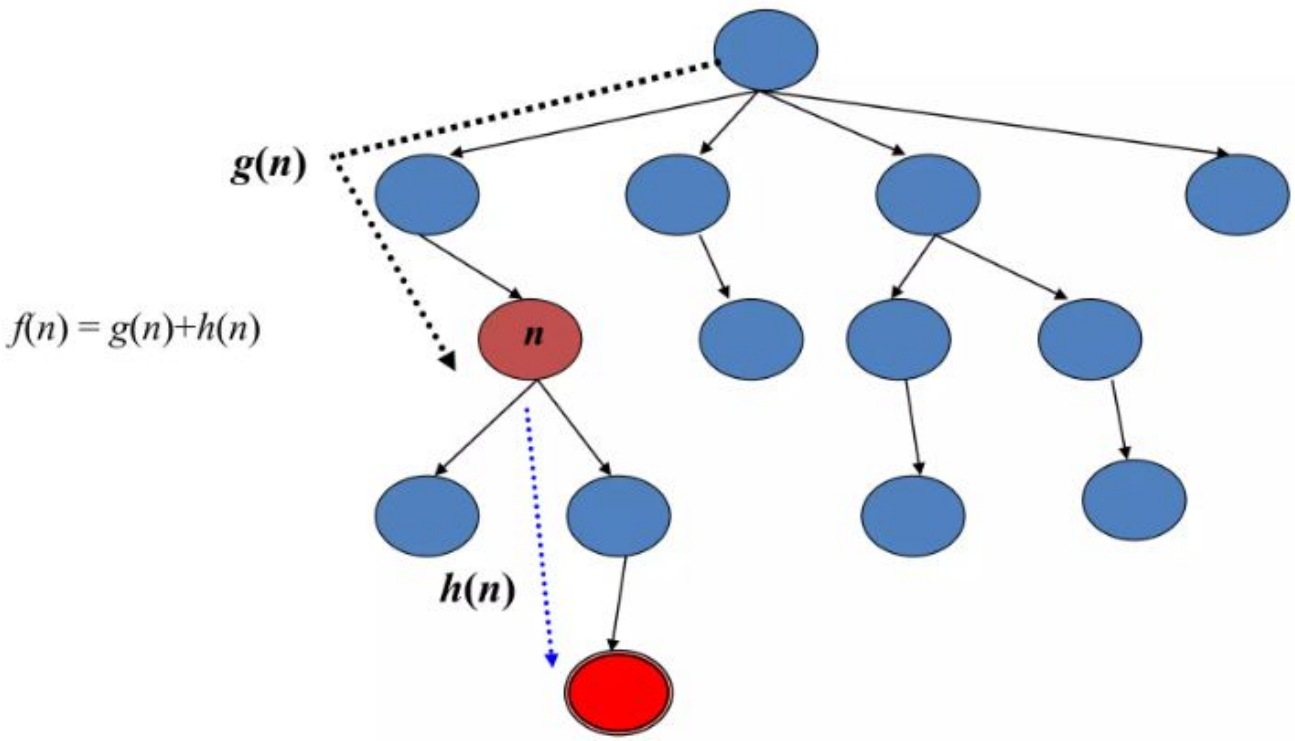
Closed list:

- Contains already evaluated nodes
- Helps avoid re-evaluating nodes
- Used to reconstruct the final path
- The algorithm continually selects the node with the lowest f(n) value from the open list, evaluates it, and moves it to the closed list until it **reaches the goal node or determines no path exists.**

Graph Search (instead of Tree Search)

```
function GRAPH-SEARCH( problem, fringe) returns a solution, or failure
    closed ← an empty set
    fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
    loop do
        if fringe is empty then return failure
        node ← REMOVE-FRONT(fringe)
        if GOAL-TEST[problem](STATE[node]) then return SOLUTION(node)
        if STATE[node] is not in closed then
            add STATE[node] to closed
            fringe ← INSERTALL(EXPAND(node, problem), fringe)
```

A* (A Star)



Admissible

- It evaluates nodes by combining g(n), the cost to reach the node, and h(n), the cost to get from the node to the goal: Evaluation function $f(n) = g(n) + h(n)$
  - g(n) = exact cost so far to reach *n* (*known distance for TSP*)
  - h(n) = estimated cost to goal from n (*informed guess about the remaining path*)
  - f(n) = estimated total cost of cheapest path through n to goal
- A* search uses an admissible heuristic:
  - $h(n) \leq h^*(n)$ where $h^*(n)$ is the true cost from n
  - Also $h(n) \geq 0$, and $h(G)=0$ for any goal G
  - e.g., $h_{SLD}(n)$ is an admissible heuristic because it doesn't overestimate the actual road distance.

Admissible heuristic

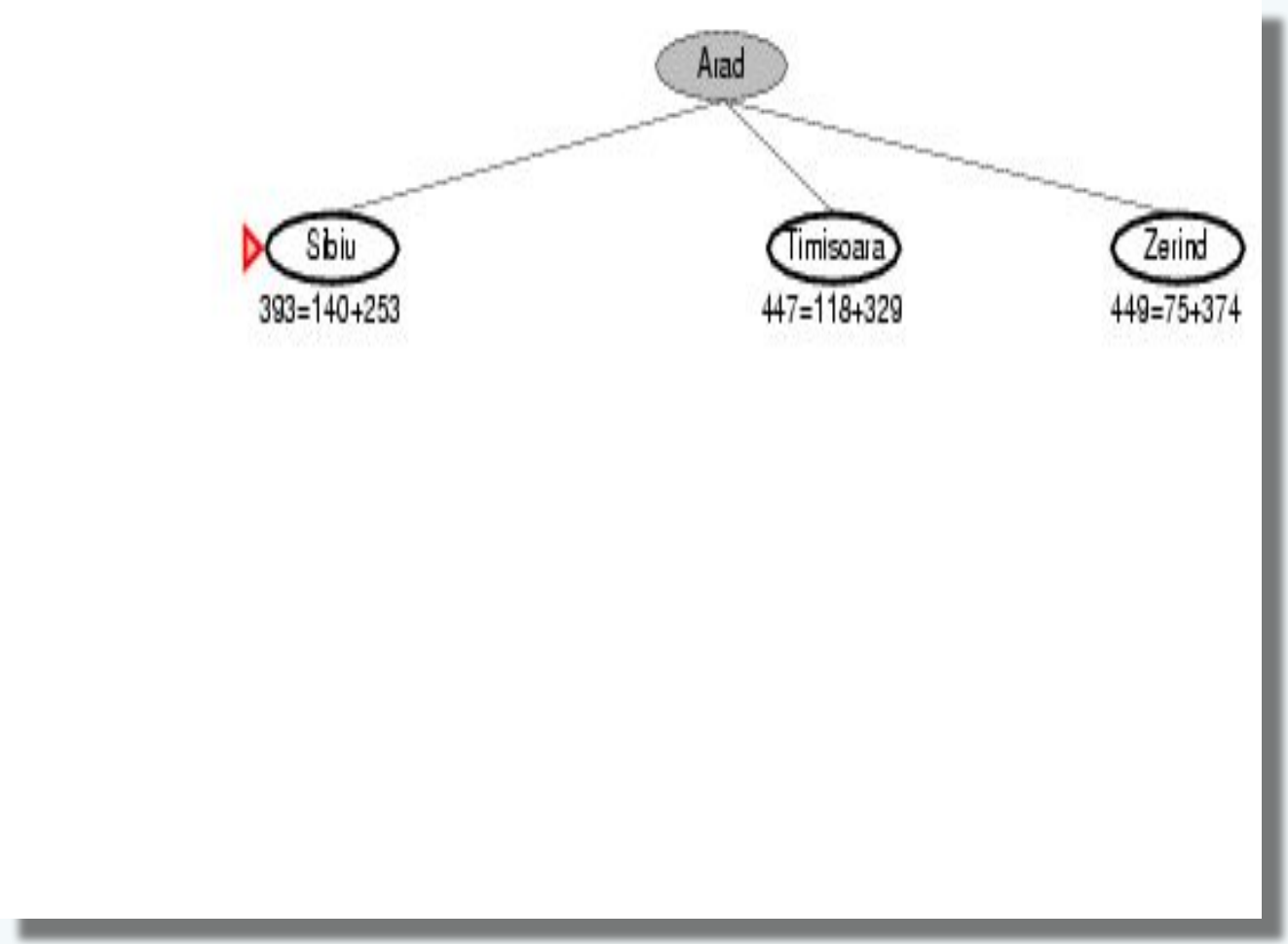
- Admissible heuristic is one that *never overestimates the cost to reach the goal.*
- *Because g(n)*is the actual cost to reach *n* along the current path, and f(n)=g(n) + h(n), we have as an immediate consequence that f(n) never overestimates the true cost of a solution **along the current path through *n***.
- Admissible heuristics are by nature **optimistic** because they think the cost of solving the problem is less than it actually is.
- An obvious example of an admissible heuristic is the straight-line distance h_{SLD} that we used in getting to Bucharest.
- Straight-line distance is admissible because the shortest path between any two points is a straight line, so the straight line cannot be an overestimate

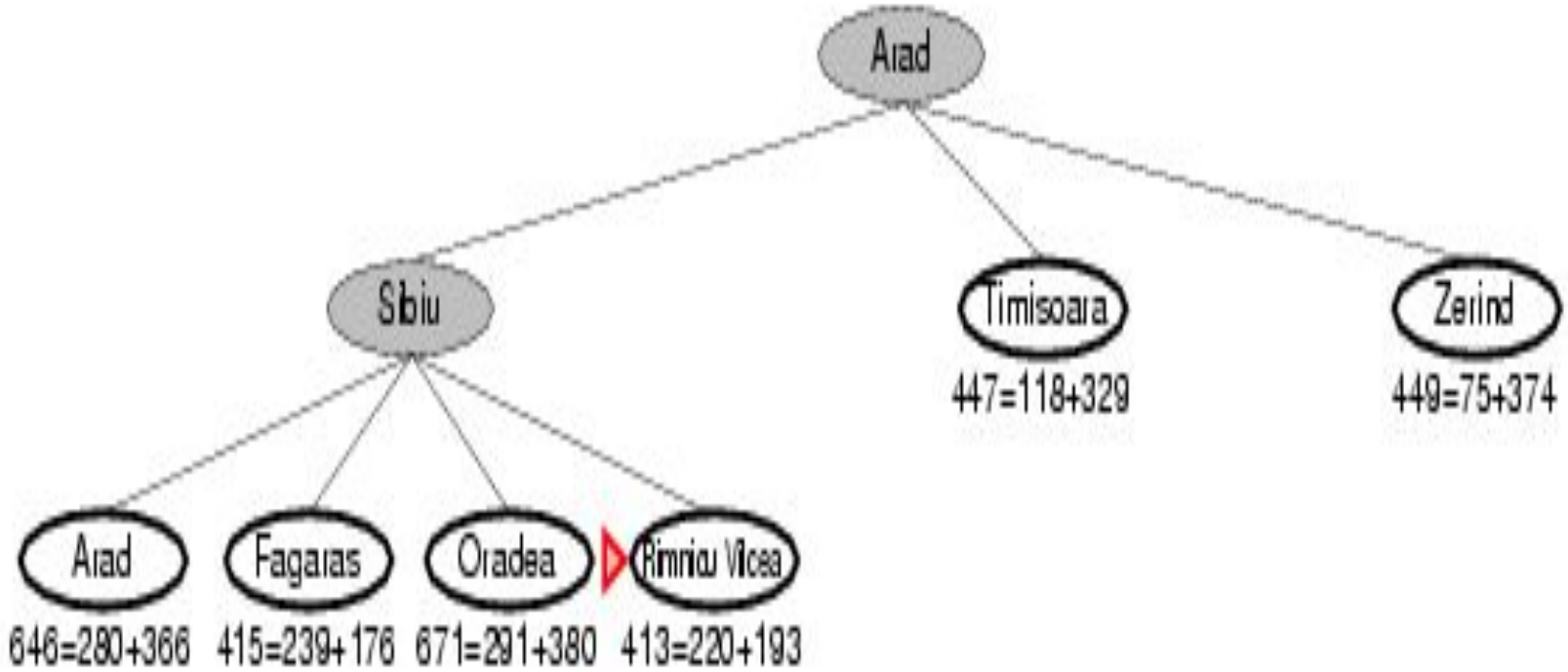
Consistency

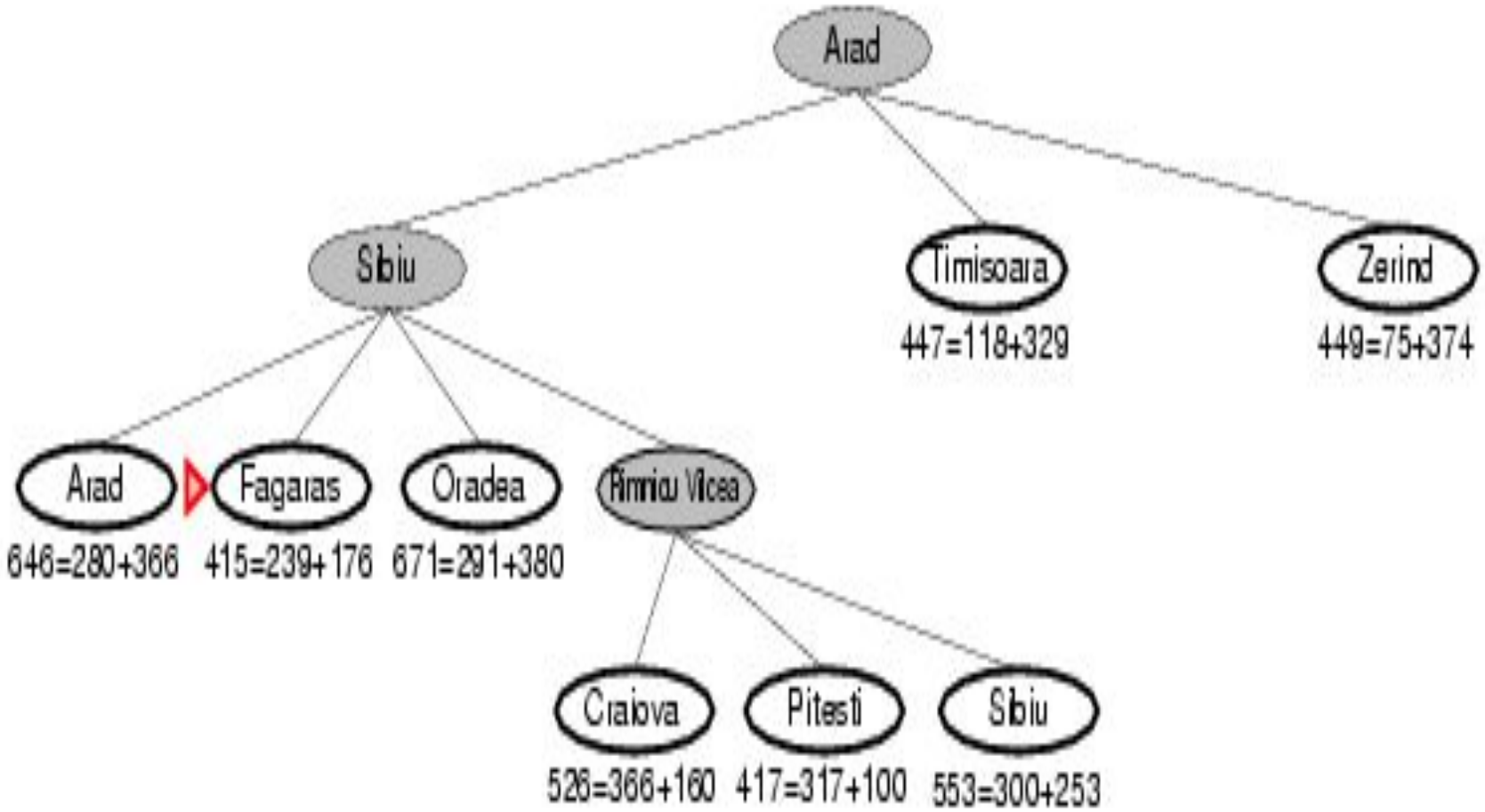
- Stronger condition called **consistency** or **monotonicity**
- A heuristic h(n) is consistent if, for every node n and every successor n′ of n generated by any action *a*, the estimated cost of reaching the goal from *n* is no greater than the step cost of getting to *n′* plus the estimated cost of reaching the goal from n′
- $h(n) \leq c(n, a, n') + h(n')$.
- If there were a route from n to G_n via n′ that was cheaper than h(n), that would violate the property that h(n) is a lower bound on the cost to reach G_n.

A* search example

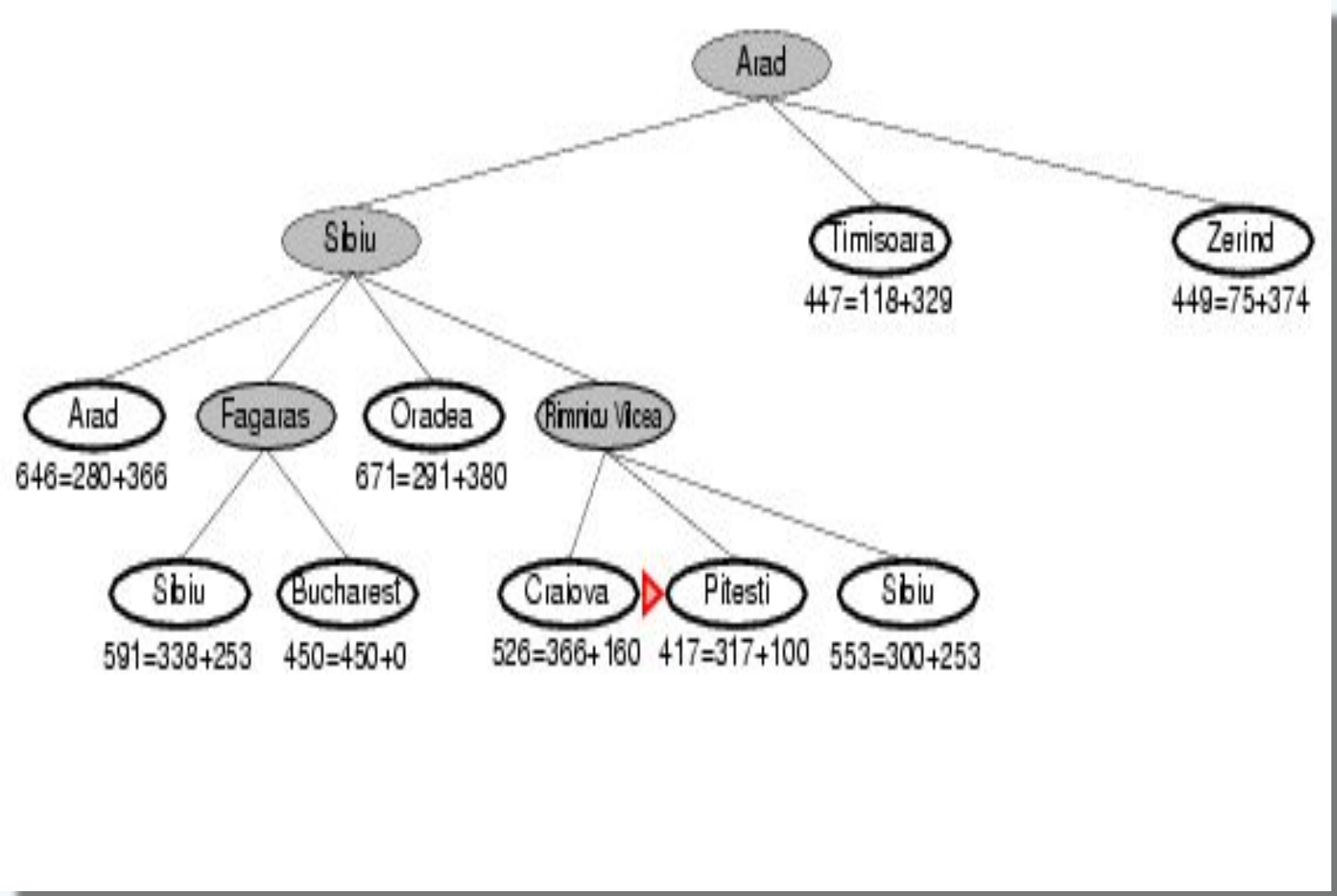
A* search example



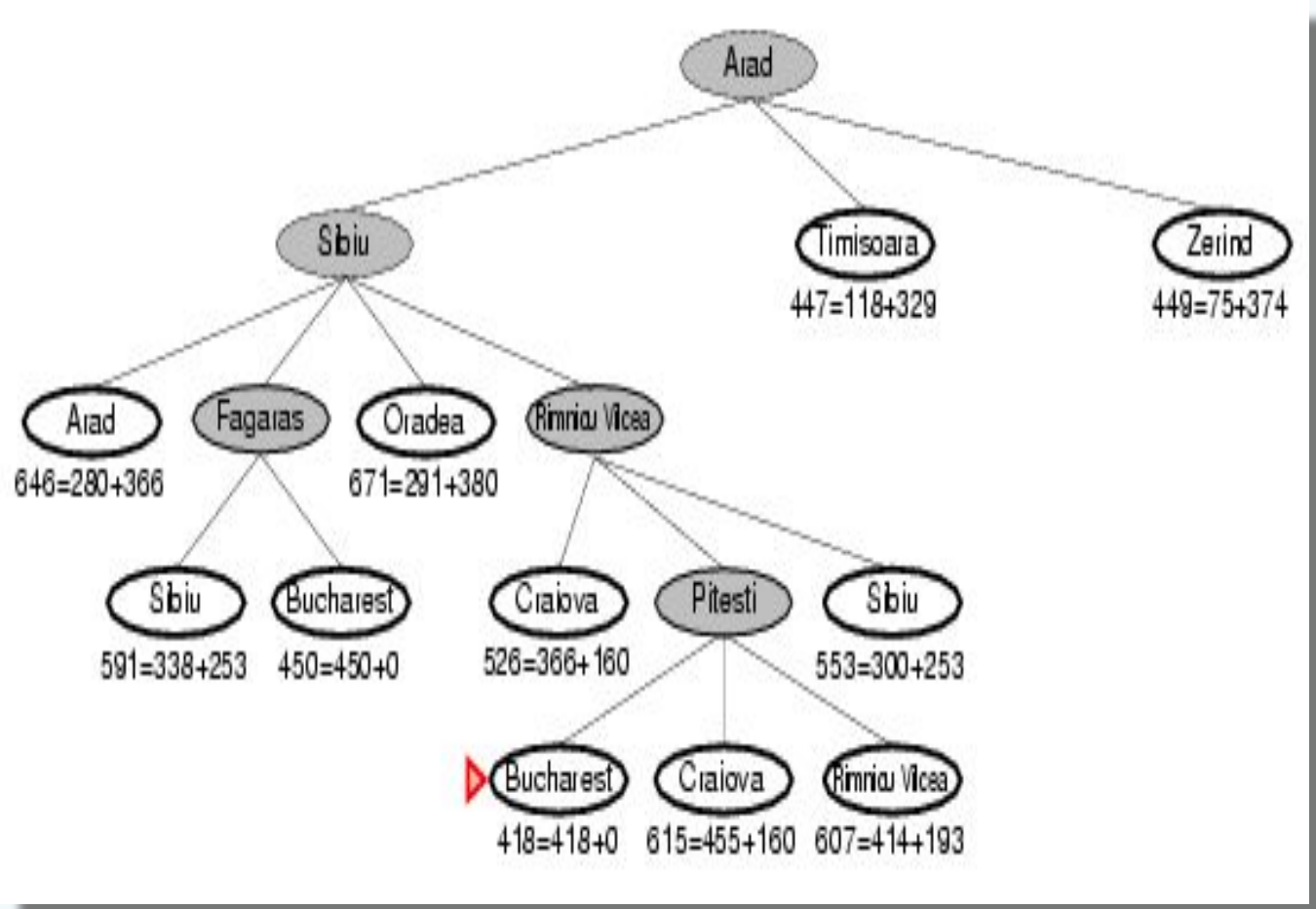

Arad
Sibiu
Timisoara
447=118+329
Zerind
449=75+374
Arad
646=280+366
Fagaras
415=239+176
Oradea
671=291+380
Rimnicu Vilcea
413=220+193


Arad
Sibiu
Timisoara
447=118+329
Zerind
449=75+374
Arad
646=280+366
Fagaras
415=239+176
Oradea
671=291+380
Rimnicu Vilcea
Craiova
526=366+160
Pitesti
417=317+100
Sibiu
553=300+253

A* search example

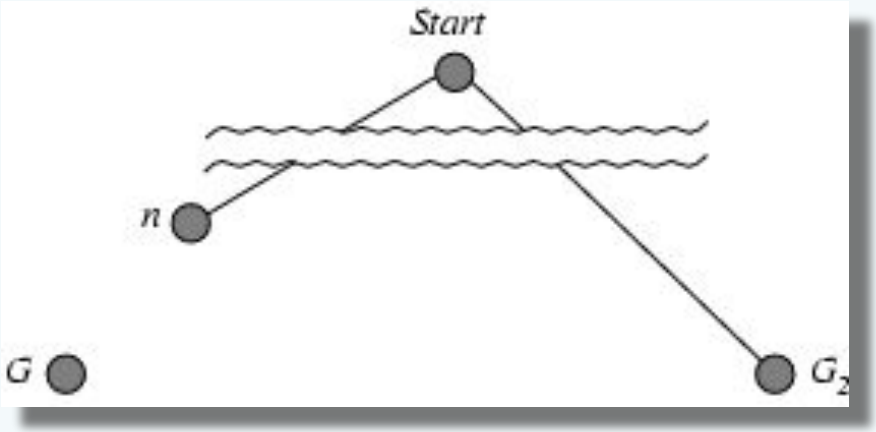


A* search example



Optimality of A* (proof)

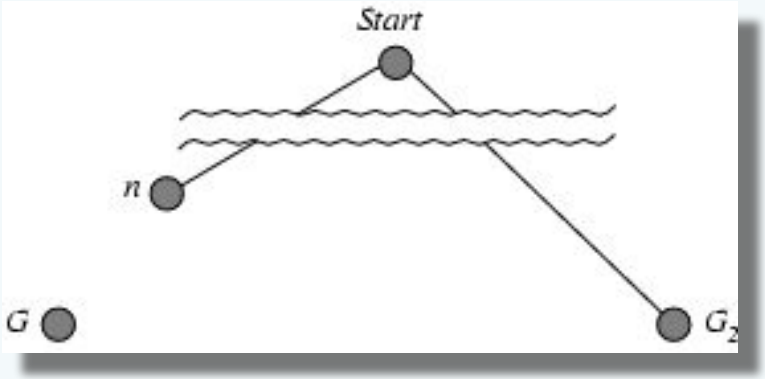
- Suppose some suboptimal goal G_2 has been generated and is in the fringe. Let n be an unexpanded node in the fringe such that n is on a shortest path to an optimal goal G.



- $f(G_2) = g(G_2)$ since $h(G_2) = 0$
- $g(G_2) > g(G)$ since G_2 is suboptimal
- $f(G) = g(G)$ since $h(G) = 0$
- Therefore, $f(G_2) > f(G)$

Optimality of A^* (proof)

- Suppose some suboptimal goal G_2 has been generated and is in the fringe. Let n be an unexpanded node in the fringe such that n is on a shortest path to an optimal goal G.

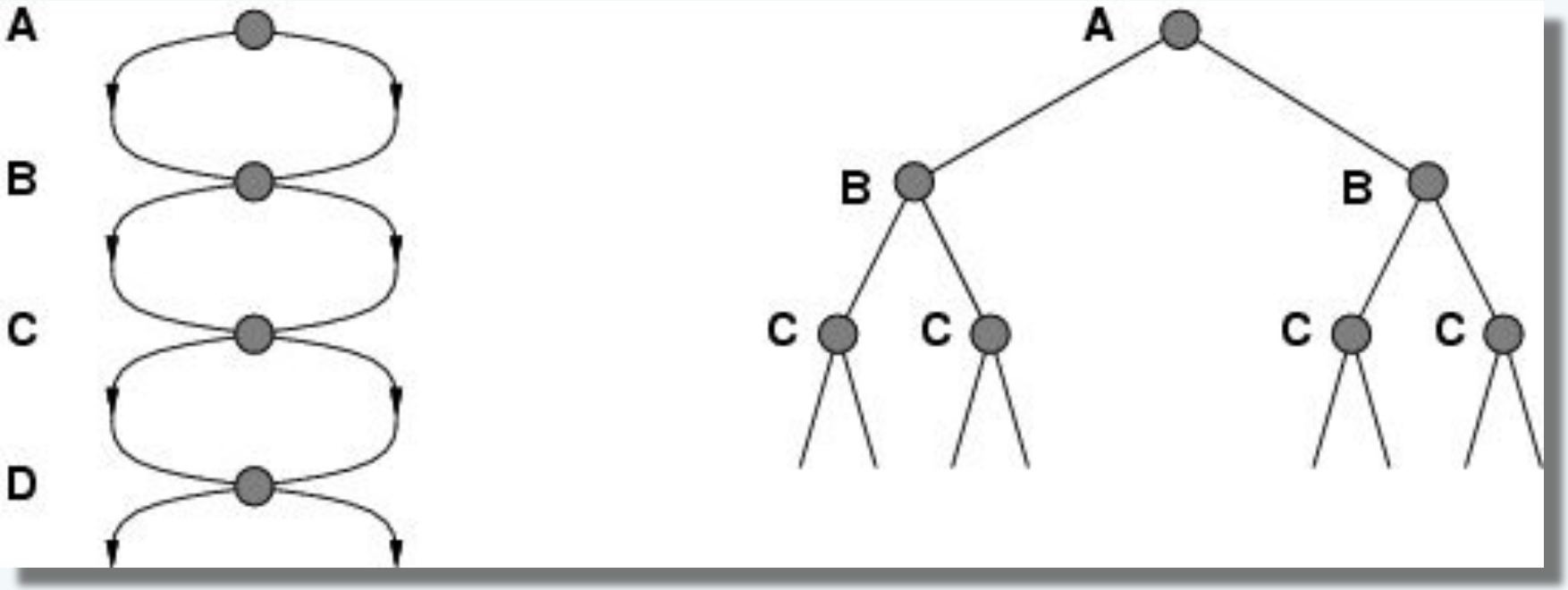


- $f(G_2) > f(G)$
- $h(n) \leq h^*(n)$ since h is admissible
- $g(n) + h(n) \leq g(n) + h^*(n)$
- $f(n) \leq f(G)$

Hence $f(G_2) > f(n)$, and A^* will never select G_2 for expansion

Problem of Repeated States

- Failure to detect repeated states can turn a linear problem into an exponential one!
  - We don't want to expand a node that has already been expanded



Graph-Search problem

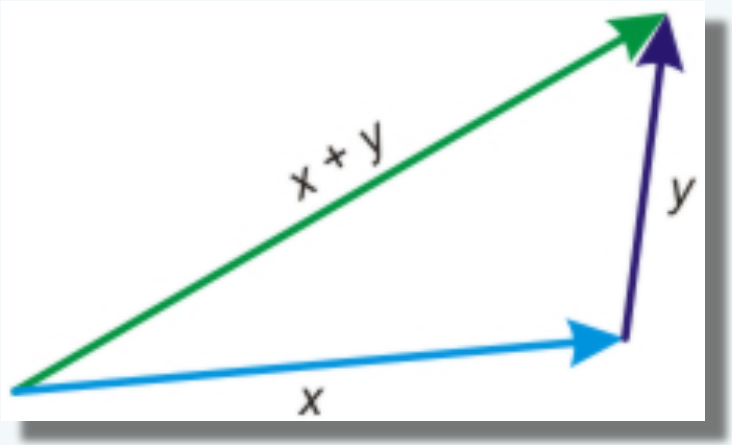
- Its very well to prune the search space by ignoring repeated states
- But the problem is that Graph-search can end up discovering sub-optimal solutions
  - Basically, a loop means that there might be more then one path to a node
  - Once we discover a path to a node, then any other paths to the same node are ignored by graph-search
  - However, it is not necessary that the first path is the optimal one
  - Hence, sub-optimal solutions can be returned.

Graph-Search problem

- Solution?
  - We can adopt a uniform-cost approach, in which we keep track of all the paths that have been currently generated
  - Then, we select only that path which has the least-cost.

Problem with A* (Proof)

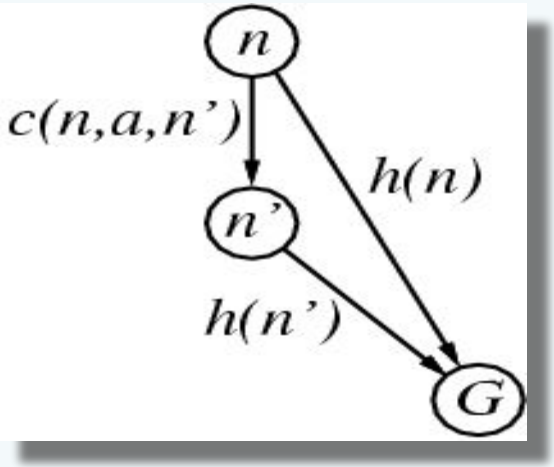
- A* can return sub-optimal solutions, if we don't apply the uniform-cost approach
- However, this is really messy and expensive
- A much better solution is to ensure that the heuristic you have selected is **consistent**



Triangle inequality

Consistent heuristics

- A heuristic is consistent if for every node *n*, every successor *n'* of *n* generated by any action *a*,
  $h(n) \leq c(n,a,n') + h(n')$



- If *h* is consistent, we have

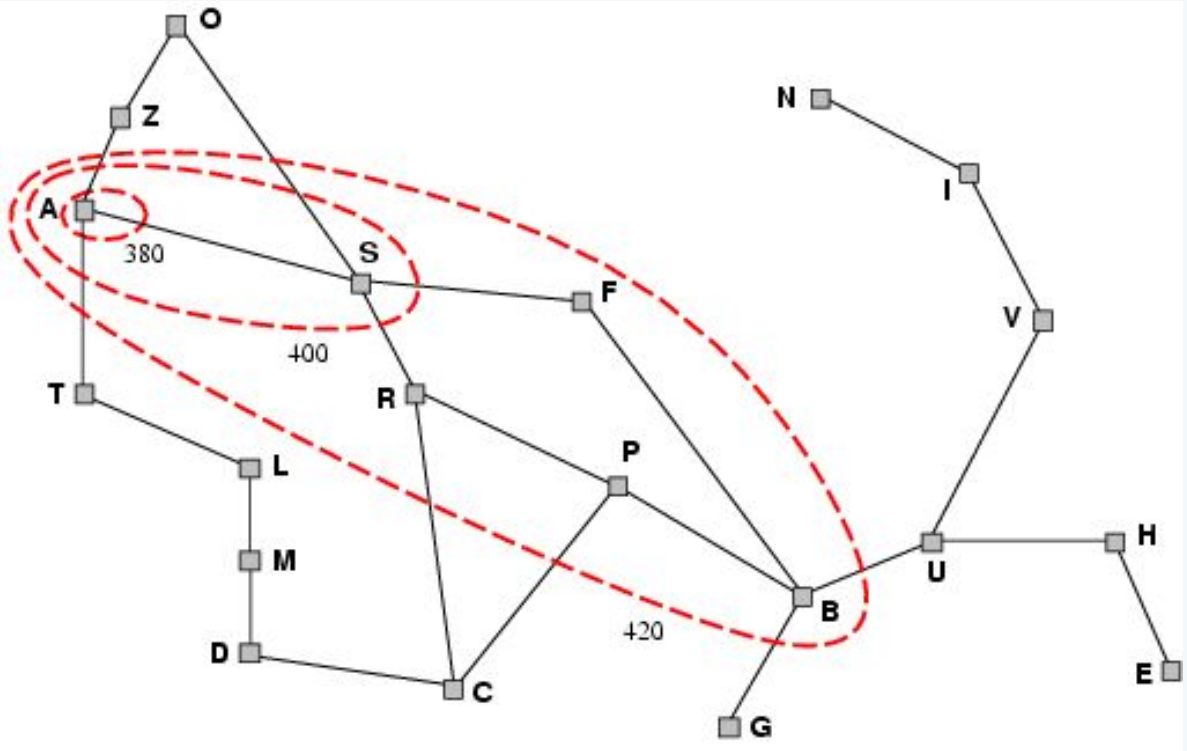
$$
\begin{aligned}
f(n') &= g(n') + h(n') \\
&= g(n) + c(n,a,n') + h(n') \\
&\geq g(n) + h(n) \\
&= f(n)
\end{aligned}
$$

- i.e., *f(n)* is non-decreasing along any path.

- Theorem: If *h(n)* is consistent, A* using `GRAPH-SEARCH` is optimal

Optimality of A*

- A* expands nodes in order of increasing *f* value
- Gradually adds "*f*-contours" of nodes
- Contour *i* has all nodes with $f=f_i$, where $f_i < f_{i+1}$



Properties of A*

- Complete? Yes (unless there are infinitely many nodes with $f \leq f(G)$)
- Time? Exponential $O(b^d)$
- Space? Keeps all nodes in memory
- Optimal? Yes

AO* Search

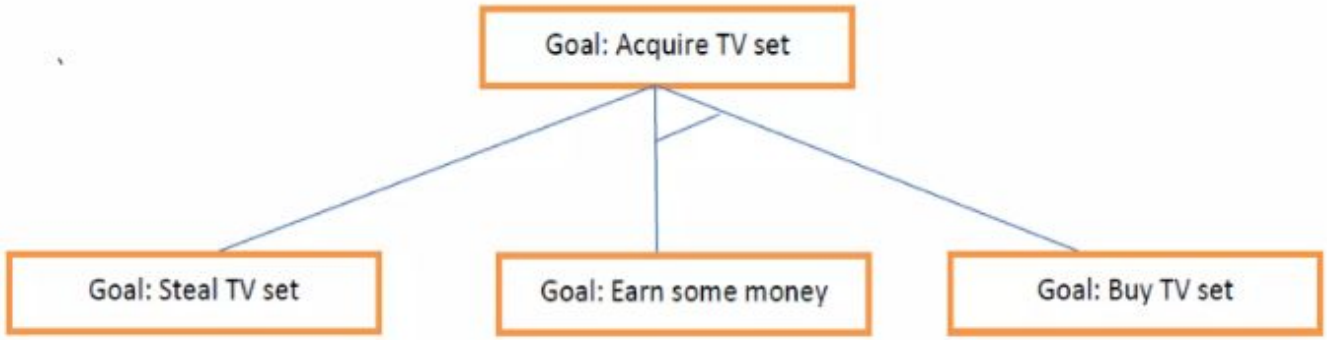
AO* SEARCH ALGORITHM

Problem Reduction In Artificial Intelligence

- AO* is informed search algorithm ,work based on heuristic
- We already know about the divide and conquer strategy, a solution to a problem can be obtained by decomposing it into smaller sub-problems.
- Each of this sub-problem can then be solved to get its sub solution.
- These sub solutions can then recombined to get a solution as a whole. That is called is Problem Reduction. AND-OR graphs or AND - OR trees are used for representing the solution.
- This method generates arc which is called as AND-OR arcs. One AND arc may point to any number of successor nodes, all of which must be solved in order for an arc to point to a solution.
- AND-OR graph is used to represent various kind of complex problem solutions.
- AO* search algo. is based on AND-OR graph so ,it is called AO* search algo.

AO* SEARCH ALGORITHM

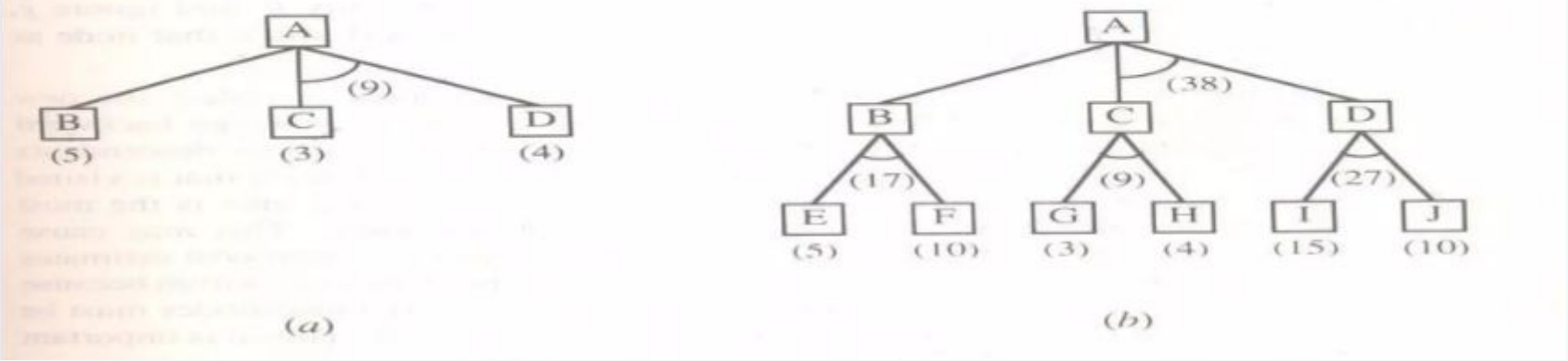
- Just as in an OR graph, several arcs may emerge from a single node, indicating a variety of ways in which the original problem might be solved.
- This is why the structure is called not simply an OR-graph but rather an AND-OR graph (which also happens to be an AND-OR tree)



A SIMPLE AND-OR GRAPH

AO* SEARCH ALGORITHM

- An algorithm to find a solution in an AND - OR graph must handle AND area appropriately.
- A* algorithm can not search AND - OR graphs efficiently.
- This can be understand from the give figure
- In figure (a) the top node A has been expanded producing two area one leading to B and leading to C-D . the numbers at each node represent the value of f ' at that node (cost of getting to the goal state from current state). For simplicity, it is assumed that every operation(i.e. applying a rule) has unit cost, i.e., each are with single successor will have a cost of 1 and each of its components.



Advantages of AO*:

- It is Complete
- Will not go in infinite loop
- Less Memory Required

Disadvantages of AO*:

- It is not optimal as it does not explore all the path once it find a solution.